

NAME: AVANCHA KIRAN KUMAR
REGNO:20233093

Assignment-8

Name: Avancha Kiran Kumar

RegNo: 20223093

1) Write a program where a parent process creates a shared memory and writes 1000 in a variable in the shared memory before creating 2 children processes A and B. After that, the parent waits for children termination and destroys the shared memory segment. Each child process performs 5 iterations. Every iteration process A adds 200, and process B adds 100 to the shared variable.

a) Could you predict the output of the program, before execution? Explain your answer.

b) Adjust the sleeping duration of one process (or both processes)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
```

```
#define SHMKEY 0x1234
```

Name: Avancha Kiron
Regno: 20233093

```
void random_sleep() {  
    int us = rand() % 100000;  
    usleep(us);  
}  
  
int main() {  
    int shmid = shmget(SHMKEY, sizeof(int), IPC_CREAT | 0666);  
    if (shmid == -1) {  
        perror("shmget");  
        exit(1);  
    }  
    int *shared_var = malloc(sizeof(int));  
    int *shared_var = (int*) shmatt(shmid, NULL, 0);  
    if (shared_var == (void*) -1) {  
        perror("shmatt");  
        exit(1);  
    }  
    *shared_var = 1000;  
    pid_t pidA = fork();  
    if (pidA == 0) {  
        srand(getpid());  
        for (int i = 0; i < 25; i++) {  
            int temp = *shared_var;  
            random_sleep();  
            temp += 200;  
            *shared_var = temp;  
        }  
    }  
}
```


Name: Avamcha Kram

RegNo: 20233093

```
shmdt(shared-var);  
exit(0);
```

}

```
pid_t pidB = fork();
```

```
if (pidB == 0) {
```

```
    srand(getpid());
```

```
    for (int i = 0; i < 5; i++) {
```

```
        int temp = *shared-var;
```

```
        random-sleep();
```

```
        temp += 100;
```

```
        *shared-var = temp;
```

```
    }  
    shmdt(shared-var);
```

```
    exit(0);
```

}

```
waitpid(pidA, NULL, 0);
```

```
waitpid(pidB, NULL, 0);
```

```
shmdt(shared-var);
```

```
return 0;
```

}

- a) NO, not precisely. This is due to race conditions:
⇒ The shared variable is updated without synchronization
⇒ Both children read, sleep, then write, possibly overlapping.

Eg:

Child A reads 1000, sleeps

Child B reads 1000, sleeps

A wakes up, writes 1200

B wakes up, writes 1100

Final value becomes 1100, not 1300.

- b) Let's modify one process to sleep significantly longer to give the other process priority.

For example, in child A

`usleep(100000);` // updated one

This gives process B the opportunity to perform more updates without conflict, which means more of B's additions may be preserved correctly.

Name: Avamcha Kiron Kumar

RegNo: 20233093

Name: Avamcha Kiron
RegNo: 20233093

2) Consider the bounded buffer problem consisting of 5 producers

and 10 consumers, and buffer of size 10 items.

Each producer performs 20 iterations. Every iteration,

a producer writes an item composed of 2 integers, values

(its 'process id' and the value of the iteration number),

into the buffer. Each consumer performs 10 iterations.

Every iteration, a consumer reads an item composed of

2 integer values from the buffer and then displays its

own 'process id' followed by 2 integers from buffer.

a) implement through semaphore-based solution

b) implement through monitor-based solution

9

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#define BUFFER_SIZE 10
```

```
#define NUM_PRODUCERS 5
```

```
#define PRODUCER_IT 20
```

```
#define PROD CONSUMER_IT 10
```


Names: Avancha Kiran Kumar

Regno: 20233053

```
typedef struct {  
    int id;  
    int it;
```

```
} Item;
```

```
Item buffer[BUFFER_SIZE];
```

```
int inco, out = 0;
```

```
sem_t empty, full, mutex;
```

```
void * producer (void * arg) {
```

```
    int id = *(int *) arg;
```

```
    for (int i = 0; i < PRODUCER_ITER; i++) {
```

```
        Item item = {id, i};
```

```
        sem_wait(&empty);
```

```
        sem_wait(&mutex);
```

```
        buffer[in] = item;
```

```
        in = (in + 1) % BUFFER_SIZE;
```

```
        sem_post(&mutex);
```

```
        sem_post(&full);
```

```
        usleep(rand() % 100000);
```

```
    }
```

```
    return NULL;
```

```
}
```


Name: Avancha Kiran Kumar
Reg No: 20233093

```
void* consumer(void* arg) {  
    int id = *(int*)arg;  
    for(int i=0; i<CONSUMER_ITER; i++) {  
        sem_wait(&full);  
        sem_wait(&mutex);  
        Item item = buffer[out];  
        out = (out+1) % BUFFER_SIZE;  
        printf("Consumer %d consumed: (%d, %d)\n",  
            id, item.id, item.it);  
        sem_post(&mutex);  
        sem_post(&empty);  
        usleep(rand() % 15000);  
    }  
    return NULL;  
}
```


Name: Avancha Kiran Kumar
Reg No: 20233093

```
int main() {  
    pthread_t producers[NUM_PRODUCERS];  
    pthread_t consumers[NUM_CONSUMERS];  
    int ids[NUM_PRODUCERS + NUM_CONSUMERS];  
    sem_t empty, full, mutex;  
    sem_init(&empty, 0, BUFFER_SIZE);  
    sem_init(&full, 0, 0);  
    sem_init(&mutex, 0, 1);  
    for (int i = 0; i < NUM_PRODUCERS; i++) {  
        ids[i] = i;  
        pthread_create(&producers[i], NULL, producer, &ids[i]);  
    }  
    for (int i = 0; i < NUM_PRODUCERS; i++) {  
        ids[i] = i;  
        pthread_create(&consumers[i], NULL, consumer, &ids[i]);  
    }  
    for (int i = 0; i < NUM_PRODUCERS; i++) {  
        pthread_join(producers[i], NULL);  
    }  
    for (int i = 0; i < NUM_CONSUMERS; i++) {  
        pthread_join(consumers[i], NULL);  
    }  
}
```



```
sem_destroy(&empty);
sem_destroy(&full);
sem_destroy(&mutex);
return 0;
```

Name: Avancha Kiran Kumar
RegNo: 20233093

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#define BUFFER_SIZE 10
#define NUM_PRODUCERS
#define NUM_CONSUMERS
#define PRODUCER_IT
#define CONSUMER_IT
typedef struct {
    int id;
    int it;
```

Item;

```
void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i=0; i<NUM_PRODUCERS; i++) {
        Item item = {id, i};
        pthread_mutex_lock(&mutex);
```

```
while (count == BUFFER_SIZE);  
pthread_cond_wait(&not_full, &mutex);
```

```
buffer[in] = item;
```

```
in = (in+1) % BUFFER_SIZE;
```

```
count++;
```

```
pthread_cond_signal(&not_empty);
```

```
pthread_mutex_unlock(&mutex);
```

```
usleep(rand() % 100000);
```

```
}  
return NULL;
```

```
}  
void* consumer(void* arg)
```

```
{  
    int id = *(int*)arg;
```

```
    for(int i=0; i<CONSUMER-1; i++)
```

```
    {  
        pthread_mutex_lock(&mutex);
```

```
        while (count == 0)
```

```
        {  
            pthread_cond_wait(&not_empty, &mutex);
```

```
        }  
        item item = buffer[out];
```

```
        out = (out+1) % BUFFER_SIZE;
```

```
        count--;
```

```
        printf("Consumer %d consumed: (%d, %d)\n",
```

```
               id, item.id, item.it);
```

Name: Anarcha Kiran

RegNo: 20233093


```

pthread_cond_signal(&notfull);
pthread_mutex_unlock(&mutex);
usleep(rand() % 150000);

```

Name : Avancha Kiran Kumar
Reg No : 20233093

```

}
return NULL;

```

```

}

```

```

int main() {

```

```

    pthread_t producers[NUM_PRODUCERS],
    consumers[NUM_CONSUMERS];

```

```

    for(int i=0; i<NUM_PRODUCERS; i++) {

```

```

        ids[i] = i;

```

```

        pthread_create(&producers[i], NULL,
            producer, &ids[i]);
    }

```

```

}

```

```

    for(int i=0; i<NUM_CONSUMERS; i++) {

```

```

        ids[i] = i;

```

```

        pthread_create(&consumers[i], NULL, consumer, NULL);
    }

```

```

}

```

```

    for(int i=0; i<NUM_PRODUCERS; i++) {

```

```

        pthread_join(producers[i], NULL);
    }

```

```

    for(int i=0; i<NUM_CONSUMERS; i++) {

```

```

        pthread_join(consumers[i], NULL);
    }

```

```

    return 0;
}

```

Problem1 output:

```
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ gedit Q11.c
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ gcc Q11.c
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ ./a.out
Process A: 1200
Process B: 1300
Process A: 1500
Process B: 1600
Process A: 1800
Process B: 1900
Process A: 2100
Process B: 2200
Process A: 2400
Process B: 2500
Final Value: 2500
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$
```

Problem2 output:

```
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ gedit Q2.c
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ gcc Q2.c
user@hp-HP-EliteDesk-800-G1-SFF:~/BISHAL(20233515)$ ./a.out
Producer 0 produced item: (0, 0)
Producer 0 produced item: (0, 1)
Producer 0 produced item: (0, 2)
Producer 0 produced item: (0, 3)
Producer 0 produced item: (0, 4)
Producer 0 produced item: (0, 5)
Producer 1 produced item: (1, 0)
Producer 1 produced item: (1, 1)
Producer 0 produced item: (0, 6)
Producer 2 produced item: (2, 0)
Consumer 0 consumed item: (0, 0)
Consumer 0 consumed item: (0, 1)
Consumer 0 consumed item: (0, 2)
Consumer 0 consumed item: (0, 3)
Consumer 0 consumed item: (0, 4)
Producer 1 produced item: (1, 2)
Producer 4 produced item: (4, 0)
Consumer 0 consumed item: (0, 5)
Consumer 0 consumed item: (1, 0)
Producer 0 produced item: (0, 7)
Producer 0 produced item: (0, 8)
Producer 2 produced item: (2, 1)
Consumer 1 consumed item: (1, 1)
Consumer 0 consumed item: (0, 6)
Producer 4 produced item: (4, 1)
```